Failure (a dead connection, and a real container) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `db_failure_and_docker_practice.py`: error handling you never saw fail is error handling you only hope works. This exercise breaks the database on purpose, then ships the app in a container and proves no secret went in with it. **If not:** the Build Task's "fake internal error → 500" and "missing env var → clear error at startup" test cases, and its Dockerfile, would all be first attempts. For Docker, put a `requirements.txt` in `practice/` (`fastapi`, `uvicorn[standard]`) and run `docker build` from there.

Solution

Story: real apps open their database connection once, at startup, like `sqlite_persistence` did — but then a bad path would crash the whole app before `/health` could even answer. Guarding the connect keeps the app up and fails only the routes that need the database. A `.dockerignore` keeps `.env` out of the image, and `docker history` proves it. **If not:** one wrong path would take down every route, and "no secret in the image" would be a hope, not a checked fact.

```
# practice/db_failure_and_docker_practice.py
import sqlite3
from fastapi import FastAPI, HTTPException

app = FastAPI()

# why: connect() itself can fail — guard it, or the app never starts
try:
    conn = sqlite3.connect("/no/such/path/runs.db", check_same_thread=False)
    conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY)")
except sqlite3.OperationalError:
    # how: None means "no database" — routes check for it below
    conn = None

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
```

```
def get_runs():
    if conn is None:
        raise HTTPException(status_code=500, detail="Something went wrong.")
    try:
        return conn.execute("SELECT * FROM runs").fetchall()
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output:

```
$ curl localhost:8000/health
{"status":"ok"}
$ curl -i localhost:8000/runs
HTTP/1.1 500 Internal Server Error
{"detail":"Something went wrong."}
```

Even though the database path is broken from the start, the app still starts and `/health` still works — only the routes that actually need the database fail, and fail cleanly.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "db_failure_and_docker_practice:app", \
     "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.env
.git
__pycache__/
*.pyc
.venv/
*.db
```

Expected output:

```
$ docker build -t my-api .
$ docker run -p 8000:8000 -e OPENAI_API_KEY=sk-test-123 my-api
$ docker history --no-trunc my-api | grep "sk-test-123"
(no output — the key was only passed with -e at run time,
 so it appears in no layer of the image)
```

`.dockerignore` stops `COPY . .` from ever picking up `.env` or `.git`, and the `grep` proves the real key value is nowhere in the image. Run this check on every image before it ships.